

Week 2 – Live Session FAQ

1. What is the main focus of Week 2 in this programme?

Week 2 moves from “getting familiar with Python” to actually *using* it to organise data and make decisions in code. You’ll work with:

- Core data types and data structures: numbers, strings, lists, tuples, sets, and dictionaries.
- Control flow: if / elif / else, logical operators, and common pitfalls like = vs == and indentation issues.
- Loops: for and while loops, range(), enumerate(), and basic loop control (break, continue, pass).
- Functions: defining your own functions, using parameters and return values, and a simple chatbot built around a `get_response()` function and a main loop.

By the end of the week, you should feel more confident reading and writing small Python programmes that make decisions and repeat tasks automatically.

2. How does Week 2 build on Week 1 and set me up for later “agentic AI” weeks?

Week 1 gave you the big-picture story of AI and let you dabble in Python and ChatGPT. Week 2 gives you the programming muscles you’ll keep using throughout the course.

This week you:

- Deepen your understanding of Python syntax instead of just copying snippets from ChatGPT.
- Practise the exact constructs later agentic frameworks rely on: lists, dictionaries, conditions, loops, and functions.
- Learn to structure logic in a way that can later be wrapped inside agents, tools, and workflows.

Later weeks assume you can comfortably read and tweak code like the chatbot example in this week—Week 2 is where you earn that comfort.

3. I’m still a beginner in Python. Will I be able to handle later agentic weeks, and how should I practise now?

Short answer: yes, if you treat Weeks 1–2 like your “practice ground” and actually code every day.

This course *does* expect you to be able to:

- Write and read small scripts with variables, lists/dicts, if statements, and loops.
- Wrap logic into functions and follow the flow of a simple programme (like the chatbot).

Think of it like learning to ride a bike: the only way to gain balance is frequent, slightly wobbly practice. Concretely, this week you should:

- Spend 20–30 minutes daily retyping and modifying examples from the slides/notebooks (change values, add new branches, tweak loops).
- Write tiny exercises:
 - A list of tasks and a loop that prints them.
 - A dictionary of products and prices, then print items above a threshold.
 - An if / elif / else chain that decides a ticket price or discount.
- Use ChatGPT smartly: ask it to explain an error or suggest 3 extra practice questions on “loops + lists”, then *code the answers yourself*.

The more small reps you do now, the easier the agentic parts will feel later.

4. What exactly are “data types” and “variables,” and why are we spending time on them?

Data types and variables are the basic grammar of Python. You’ll use them every single week.

- **Data types** are categories of values:
 - Numbers (int, float, complex)
 - Text (str)
 - Collections (lists, tuples, sets, dictionaries)
- **Variables** are names you give to those values so you can reuse them later in the code.

In Week 2, you’ll:

- Practise declaring variables and converting between types (e.g., string to integer).
- See how choosing the right data type makes later logic and loops much cleaner.

If you get this part solid, everything else (conditions, loops, functions, agents) becomes far less painful.

5. When should I use a list, tuple, set, or dictionary?

This week explicitly compares these four structures and shows when each is a better fit.

Rough mental model:

- **List:**
 - Ordered, changeable, allows duplicates.
 - Use for sequences you'll iterate over or edit (e.g., a to-do list).
- **Tuple:**
 - Ordered, *not* changeable.
 - Good for fixed records like a (latitude, longitude) pair.
- **Set:**
 - Unordered, unique elements, fast membership tests.
 - Useful for removing duplicates or checking "is this item present?".
- **Dictionary:**
 - Key-value pairs, great for lookups (e.g., user_id → profile).

In the live session, you'll see examples and short exercises that force you to choose between them so the differences stick.

6. What do "control flow" and "conditions" look like in real code?

Control flow is just "which path does my code take?" based on some condition. Week 2 covers:

- if, if-else, and if-elif-else patterns with real examples.
- Nested if statements and how to avoid making them unreadable.
- Logical operators (and, or, not) to build more nuanced rules.

You'll practice things like:

- "If age < 12 charge child price, elif age < 60 charge adult price, else charge senior price."
- "If the user typed 'bye', terminate the chatbot loop."

This is the same kind of branching you use in daily life ("if traffic is bad, take the metro; otherwise drive"), just written in code.

7. What are loops, and how deep do we go with them this week?

Loops are just “repeat this until I’m done” written in Python. Week 2 focuses on:

- **for loops** over sequences like lists or strings.
- **while loops** that repeat while a condition is True.
- **Loop control statements:** break, continue, and pass.
- **range()** for numeric sequences and **enumerate()** when you need index + value.

You won’t be doing heavy algorithmic work; you’ll use loops to:

- Process a list of items (e.g., printing all tasks).
- Repeatedly prompt a user until they say “bye” in the chatbot.

The goal is comfort, not speed-coding.

8. What are functions, and how much of the syntax do I need to remember?

Functions are named blocks of code that perform a specific task. Week 2 teaches you to:

- Distinguish between built-in and user-defined functions.
- Define your own functions with parameters and return values.
- Use default parameters, *args and **kwargs, and understand variable scope (local vs global).
- Apply simple best practices: good names, short functions, docstrings, and minimal side effects.

You don’t need to memorise every corner case. Focus on being able to:

- Read a function and say what it does.
 - Write small, clear functions like `get_ticket_price(age)` or `get_response(message)`.
-

9. What will the chatbot example actually do? Are we building a full production chatbot?

No, this is a small, controlled exercise—not a production system. The chatbot is there to tie together conditionals, loops, and functions.

In the live session, you’ll see how to:

- Use a while loop to keep the chatbot running for multiple user inputs.
- Use if-elif-else to pick responses based on what the user types.
- Encapsulate reply logic in a `get_response()` function and call it from the main loop.

You will **not** be:

- Deploying the chatbot to a website or messaging app.
- Integrating external APIs or large models inside it.

Those are later-stage concerns. For now, the goal is: “Can I read and control a conversational loop in Python?”

10. What is the Chatbot Debugging Exercise, and how does it connect to the live session?

The Chatbot Debugging Exercise gives you a partly broken chatbot and asks you to fix it. The idea is to:

- Practise spotting common Python errors in loops and conditions.
- Use ChatGPT to interpret error messages and suggest fixes, then decide which suggestions are actually correct.
- Strengthen your understanding of how user input flows through the chatbot logic.

It mirrors the concepts from the live demo—while loops, if / elif / else, and a `get_response()` function—so you’re not seeing anything “new”, just stress-testing what you’ve already learned.

11. How will we use ChatGPT or other AI tools in Week 2 if this week is about Python?

AI tools are present, but in a supporting role. In this week, ChatGPT is mainly used to:

- Explain error messages and suggest possible fixes.
- Provide alternative implementations you can compare against your own.
- Help you think through logic in plain language before you code it.

However, the emphasis is on *you* understanding the code:

- You’ll still run, modify, and debug the Python yourself.
- The goal is not “ask ChatGPT to write everything”, but “use it as a debugging and learning partner”.

This mindset is crucial for later weeks when you orchestrate agents rather than copy-pasting code unthinkingly.

12. Do I need to complete all the Week 2 pre-reads before the live session?

They're strongly recommended, but not a gate. The pre-reads are short tutorials on:

- Variables and data types.
- Lists, tuples, sets, and dictionaries.
- if...else statements, loops, and functions.

Use them strategically:

- If you're newer to Python, skim once and try a few of the examples so the live session doesn't feel like "first contact".
- If you're more experienced, use them as a quick refresher and reference.

Either way, having those pages open during the session can save you time when something looks unfamiliar.

13. Will we use advanced Python libraries like NumPy, pandas, or web frameworks in Week 2?

No. Week 2 stays close to core Python. You'll work with:

- Built-in types and data structures (numbers, strings, lists, sets, dictionaries).
- Core language constructs: conditions, loops, and user-defined functions.

Libraries like NumPy, pandas, or web frameworks require additional setup and concepts. They're helpful in data science and production systems, but they sit *on top of* the basics you're practising now. This week is deliberately focused on those basics.

14. What kinds of mistakes should I expect to make, and how will we handle them in class?

You're *meant* to make mistakes this week—that's how you learn. The deck even calls out common issues:

- Using = instead of == in conditions.
- Forgetting indentation or misaligning code blocks.
- Over-nesting if statements and making logic hard to read.
- Confusing boolean checks like if x vs if x is None.

In the live session and exercises, you will:

- Run into some of these errors on purpose.
- Use error messages plus ChatGPT explanations to fix them.
- Refactor clumsy logic into clearer if / elif / else or loop structures.

Treat each bug as “another rep” in learning the language, not as a failure.

15. If I start feeling overwhelmed during Week 2, what should I focus on first?

If everything feels overwhelming, narrow your focus. For this week, the *high-leverage* pieces are:

- **Lists and dictionaries** – read, write, and loop over them.
- **if / elif / else** – understand how branching decisions are written.
- **for and while loops** – be able to trace what runs and when it stops.
- **Simple functions** – define one, pass arguments, and use the return value.

If you can:

- Understand the chatbot example end-to-end, and
- Write or adapt one small function using conditions and loops,

you’re in good shape for what the rest of the programme will expect from your Python skills.